

UNIVERSIDAD NACIONAL
AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA



LAB. COMPUTACION GRAFICA E INTERACCION HUMANO-COMPUTADORA

MANUAL TÉCNICO

Proyecto Final - Tema: Cultural

Altar de muertos en Cementerio

DATOS ACADÉMICOS

Integrantes	Echeverria Goicochea, Erick Isaac [424153520] Cruz Miranda, Yasser Vladimir [424039512]
Profesor	Espinoza Urzúa, Edén
Grupo	4
Semestre	2027-1
Repositorio	`https://github.com/VvyasservV/Ofrenda-dia-de-muertos-laboratorio-de-CG`

Fecha de entrega: 09/11/2026

Índice

1 Introducción **2**

2 Objetivos **2**
2.1 Objetivo general 2
2.2 Objetivos específicos 2

3 Alcance y limitantes **2**
3.1 Alcance del proyecto 2
3.2 Limitantes del proyecto 3

4 Diseño y arquitectura del software **3**
4.1 Arquitectura general 3
4.2 Diagrama de flujo 3
4.3 Estructura de archivos y carpetas 4
4.4 Metodología de software aplicada 5
4.5 Planificación y cronograma (Diagrama de Gantt) 6

5 Modelado y recursos gráficos **6**
5.1 Descripción de los objetos modelados 6
5.2 Texturas y materiales 10
5.3 Iluminación y ambientación 10

6 Documentación del código fuente **11**
6.1 Inicialización de OpenGL 11
6.2 Sistema de renderizado 12
6.3 Cámara y perspectiva 12
6.4 Manejo de entradas del usuario 13
6.5 Funciones principales 13

7 Sistema de animaciones **14**
7.1 Animación sencilla 1: viento en el papel picado (tecla 1) 14
7.2 Animación sencilla 2: banda tocando (tecla 2) 14
7.3 Animación sencilla 3: reja de la entrada (tecla 3) 14
7.4 Animación compleja 1: niño que saluda (tecla 4) 14
7.5 Animación compleja 2: mariposas monarca (tecla 5) 15

8 Resultados **16**
8.1 Evidencias del resultado final 16

9 Conclusiones **16**

10 Repositorio del proyecto **17**

1 Introducción

El presente proyecto consiste en la recreación de un panteón mexicano durante la celebración del Día de Muertos, tomando como referencia una escena tradicional que incluye tumbas, flores de cempasúchil, velas y una ofrenda. El desarrollo se realizó utilizando OpenGL 3.3 y el código base proporcionado durante las clases del laboratorio, integrando distintos elementos de computación gráfica que se trabajaron a lo largo del curso. Para construir la escena se utilizaron modelos 3D, texturas, diferentes fuentes de iluminación y técnicas de animación, además de implementar controles que permiten al usuario recorrer el entorno e interactuar con algunos de sus elementos. De este modo, se buscó crear una escena tridimensional que representara este espacio de forma detallada y que, al mismo tiempo, permitiera aplicar los conocimientos adquiridos durante las clases del laboratorio.

2 Objetivos

2.1 Objetivo general

Recrear en 3D, con OpenGL 3.3 y el código base del laboratorio, un panteón mexicano en la noche de Día de Muertos con una ofrenda tradicional, a partir de imágenes de referencia, aplicando lo aprendido en el curso: carga de modelos, texturizado, iluminación, modelado jerárquico, animación e interacción con el usuario.

2.2 Objetivos específicos

- Modelar y texturizar al menos siete objetos de la imagen de referencia con la mayor fidelidad posible.
- Iluminar la escena de noche con el modelo de Phong (luna, velas y linterna), agregar un brillo de borde (Fresnel), niebla y un skybox nocturno.
- Programar cinco animaciones por código que tengan sentido en el contexto (tres sencillas y dos complejas), basadas en las prácticas de Animación básica, Máquina de estados y KeyFrames.
- Permitir que el usuario recorra la escena con teclado y ratón y que prenda y apague las luces y animaciones.
- Entregar un ejecutable que no dependa de la carpeta Debug.

3 Alcance y limitantes

3.1 Alcance del proyecto

El proyecto incluye:

- Un recorrido libre en primera persona por el panteón, con la cámara del código base (Camera.h).
- La escena completa: barda y reja de la entrada, capilla, terrazas con escaleras, 20 tumbas de distintos tipos (entre ellas cuatro mausoleos), cruces, un ángel, angelitos, árboles dentro y fuera de la barda, la ofrenda de varios niveles con todos sus elementos, coronas, tapetes de flores, papel picado, guirnaldas, caminos de cempasúchil, 273 velas, 48 personas y una banda de siete músicos. En total son 49 modelos distintos y 1,208 objetos colocados.
- Cinco interacciones con la iluminación: velas, linterna, intensidad de la luna, brillo de borde y niebla.
- Cinco animaciones por código activadas con las teclas 1 a 5.
- Ejecutable en build/, manual de usuario, manual técnico, video.

3.2 Limitantes del proyecto

El proyecto tiene las siguientes limitantes técnicas:

- **Versión de assimp.** El assimp incluido en el código base es antiguo. Los modelos se exportan a OBJ, que sí lee sin problemas.
- **Rendimiento.** Con más de 1,500 objetos y 24 luces puntuales por píxel, una tarjeta integrada puede bajar de cuadros por segundo. Se agregaron optimizaciones (sección 6.2), pero la niebla y la cantidad de velas siguen siendo lo más costoso.
- **Tiempo de carga.** Los modelos se cargan al iniciar, por lo que el programa tarda algunos segundos en mostrar la escena.
- **Tamaño de los modelos.** El proyecto pide que cada objeto pese menos de 100 MB, el más pesado (los pétalos del camino) pesa unos 8 MB.

4 Diseño y arquitectura del software

4.1 Arquitectura general

El proyecto parte de la práctica *Fuentes de luz* (`fuentesDeLuz.cpp`): la ventana, el ciclo principal, la cámara, el manejo del teclado y del ratón, y el shader de Phong con luz direccional, luces puntuales y una linterna son los del laboratorio. Sobre esa base, todo el programa vive en un solo archivo (`main.cpp`), el cual coordina la carga de modelos, la escena, las luces, las animaciones y la interacción, y los shaders están en archivos aparte. La escena no se escribe en código: se lee de archivos de datos (sección 4.3), de modo que cada modelo se carga una sola vez y se dibuja en todas sus posiciones.

La tabla 1 resume las herramientas y tecnologías usadas.

Herramienta	Uso
C++ y Visual Studio 2022	Lenguaje y entorno de desarrollo (Release, Win32)
OpenGL 3.3 (perfil core) y GLSL 330	Dibujo de la escena y shaders
GLFW	Ventana, teclado y ratón
GLEW	Funciones de OpenGL
GLM	Matrices y vectores (transformaciones)
assimp	Carga de los modelos OBJ (en `Model.h` del código base)
SOIL2	Carga de texturas PNG y de las caras del skybox
Blender 5.2	Modelado, texturizado y exportación a OBJ
Git, GitHub y GitHub Desktop	Control de versiones

Tabla 1: Herramientas y tecnologías.

4.2 Diagrama de flujo

La figura 1 muestra el flujo del programa: en morado la inicialización y en naranja lo que se repite en cada cuadro.

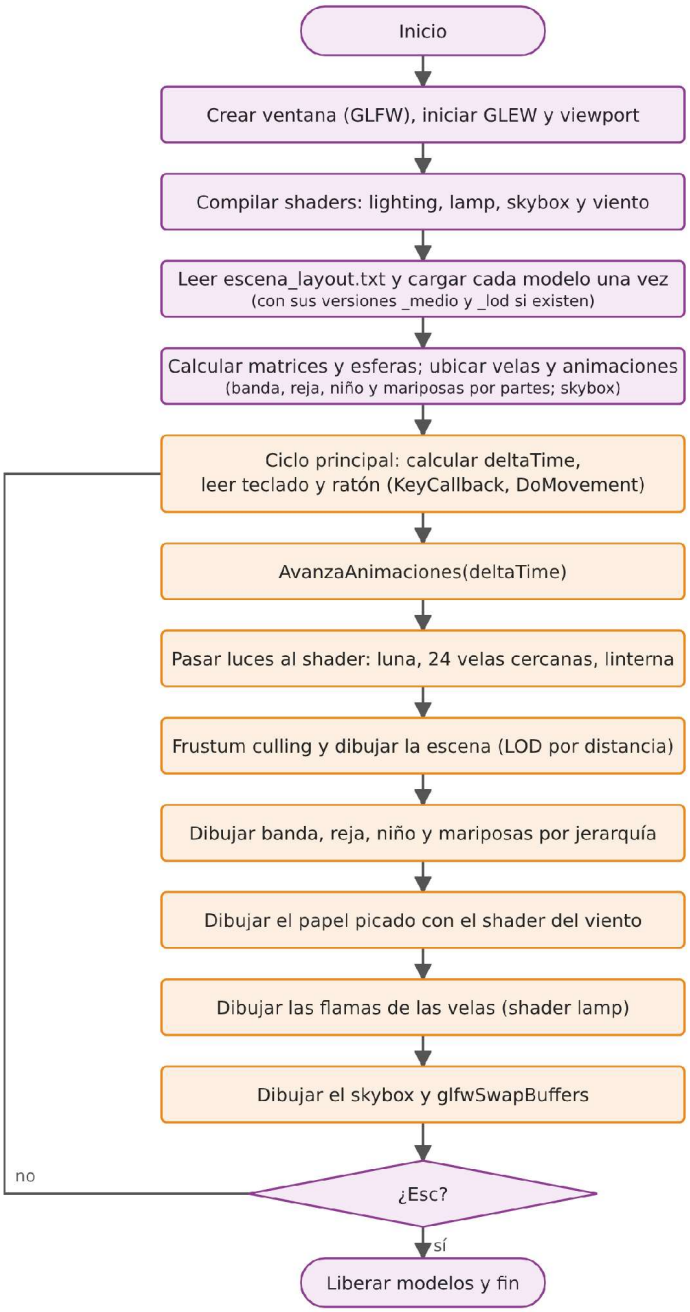


Figura 1: Diagrama de flujo del programa.

4.3 Estructura de archivos y carpetas

Ruta	Contenido
`ProyectoFinal.sln` / `.vcxproj`	Solución de Visual Studio (Release | Win32)
`src/main.cpp`	Programa principal: escena, luces, animaciones e interacción
`include/`	Código base del laboratorio: `Shader.h`, `Camera.h`, `Mesh.h`, `Model.h`, SOIL2, `stb_image.h`
`shaders/lighting.vs/.frag`	Phong (luna, 24 velas, linterna) + Fresnel + niebla + tinte + transparencia

Ruta	Contenido
`shaders/viento.vs`	Igual que `lighting.vs`, pero mueve el papel picado (animación 1)
`shaders/lamp.vs/.frag`	Flamas de las velas (color sólido)
`shaders/skybox.vs/.frag`	Cubo del cielo nocturno con niebla
`assets/escena_layout.txt`	Un renglón por objeto de la escena
`assets/animaciones_layout.txt`	Lugar de la banda y del niño que saluda
`assets/skybox/`	Seis caras del cielo (right, left, top, bottom, front, back)
`assets/models/<modelo>/`	`<modelo>.obj`, `.mtl` y texturas PNG (y `_medio.obj`, `_lod.obj` si tiene)
`assets/models/banda/`	Siete músicos por partes con su `pivotes.txt` y `banda_layout.txt`
`assets/models/persona_nino_partes/`	Niño por partes (animación 4)
`assets/models/reja/`	Las dos hojas de la reja con el origen en la bisagra (animación 3)
`assets/models/mariposa/`	Cuerpo y alas de la mariposa monarca (animación 5)
`external/`	GLEW, GLFW, assimp y glm
`build/`	`ProyectoFinal.exe` y sus .dll
`docs/`	Manuales e imágenes de referencia

Tabla 2: Estructura del repositorio.

4.4 Metodología de software aplicada

Se usó una metodología de prototipos evolutivos con entregas incrementales: primero se hizo un prototipo que solo cargaba los modelos, y en cada iteración se le agregó una parte nueva, se probó y se corrigió antes de pasar a la siguiente.

Fase	Qué se hizo
1. Requisitos y planeación	Elegimos el tema (panteón en Día de Muertos con una ofrenda), las imágenes de referencia y los objetos a recrear. Revisamos el PDF del proyecto y repartimos el trabajo por días.
2. Diseño	Definimos la distribución del panteón (entrada, camino, tumbas a los lados y ofrenda frente a la capilla), las luces y las cinco animaciones. Diseñamos el formato de `escena_layout.txt` para colocar los objetos.
3. Prototipo inicial	Estructura del repositorio, código base del laboratorio, shaders y skybox; después los primeros modelos texturizados y una escena con luces.
4. Iteraciones	Se agregaron por partes las terrazas, tumbas y velas; la ofrenda, coronas, tapetes y papel picado; las personas; y una por una las cinco animaciones. Después de cada parte se compiló, se recorrió la escena y se corrigieron objetos encimados, flotando o sin textura.
5. Pruebas	Se revisó cada tecla, cada animación y el rendimiento, y optimizamos el dibujo de la escena. Al final quitamos objetos para que la escena se viera menos saturada y reacomodamos tumbas y personas para que no quedaran huecos.
6. Entrega	Ejecutable en `build/`, manuales, video y exposición.

Fase	Qué se hizo

Tabla 3: Fases de la metodología.

4.5 Planificación y cronograma (Diagrama de Gantt)

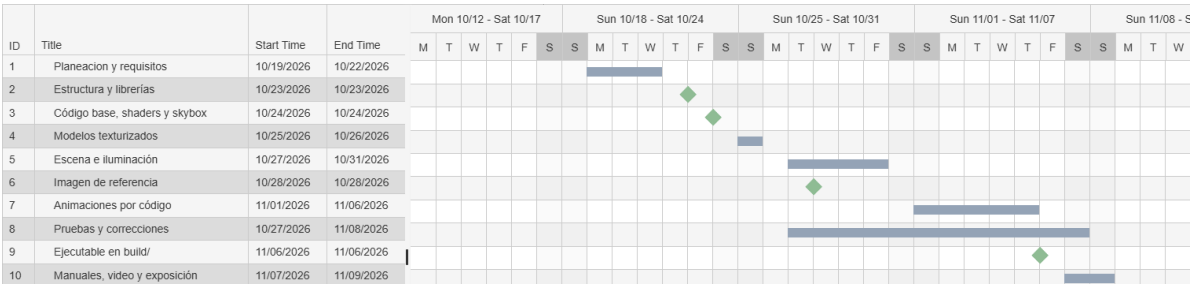


Figura 2: Diagrama de Gantt del proyecto. Las pruebas se hicieron a lo largo de todo el desarrollo.

Fecha	Responsable	Avance subido a GitHub
23 oct	Yasser	Estructura del proyecto y librerías
24 oct	Erick	Código base, shaders de iluminación y skybox
25 oct	Yasser	Escenario, capilla y primeros modelos
26 oct	Erick	Catrina, calaveritas, portarretratos y cempasúchil
27 oct	Yasser	Escena del panteón con luces y niebla
28 oct	Erick	Imagen de referencia
29 oct	Yasser	Terrazas, tumbas y velas
30 oct	Erick	Ofrenda, coronas, tapetes y papel picado
31 oct	Yasser	Personas
1 nov	Erick	Animación 1: viento en el papel picado
2 nov	Yasser	Animación 2: banda
3 nov	Erick	Animación 3: reja
4 nov	Yasser	Animación 4: niño que saluda
6 nov	Erick	Animación 5: mariposas
6 nov	Yasser	Ejecutable en `build/`
7 nov	Erick	Manual de técnico
8 nov	Yasser, Erick	Cambios en la escena para optimizarla, Manual usuario y README final

Tabla 4: Avances diarios subidos al repositorio.

5 Modelado y recursos gráficos

5.1 Descripción de los objetos modelados

Los objetos se modelaron y texturizaron en Blender y se exportaron a OBJ con sus texturas PNG. Los objetos de la imagen de referencia que se recrean son los de la tabla 5.

Objeto	Carpeta	Descripción
Ofrenda	`ofrenda`	Ofrenda de varios niveles con manteles rosas, cajas laterales, banca y macetas.

Objeto	Carpeta	Descripción
Vela	`vela`, `flama`	Vela de cera con flama, cada una es una luz puntual.
Calaverita de azúcar	`calaverita`	Calavera con betún de colores.
Portarretratos	`portarretratos`	Marco de madera con foto.
Cempasúchil	`flor_cempasuchil`, `petalos_camino`	Flor con tallo y hojas, y caminos de pétalos.
Papel picado	`papel_picado`	Tiras de papel de colores con huecos (transparencia).
Pan de muerto	`pan_de_muerto`	Pan con huesitos y azúcar.
Platillos	`platillos_altar`	Platos, jarros, frutas y comida de la ofrenda.
Florero	`florero`	Florero de porcelana blanca con cempasúchil.
Catrina	`catrina`	Catrina con vestido negro floreado y sombrero con plumas.

Tabla 5: Objetos principales de la recreación.

Para la ambientación del panteón se agregaron: escenario con barda y reja, capilla, terrazas con escaleras, tumbas (tres tipos), cruces, ángeles, árboles, copaleros, coronas (seis tipos), tapetes de flores (siete diseños), guirnaldas, personas (doce variantes de hombre, mujer, niño y niña), la banda y las mariposas.

Además de la imagen de la ofrenda completa (figura 9), para cada uno de los diez objetos principales se buscó una foto real que se pareciera a lo que se iba a modelar y se usó como guía de forma, proporción y color. En la figura 3 está cada foto junto a su modelo.

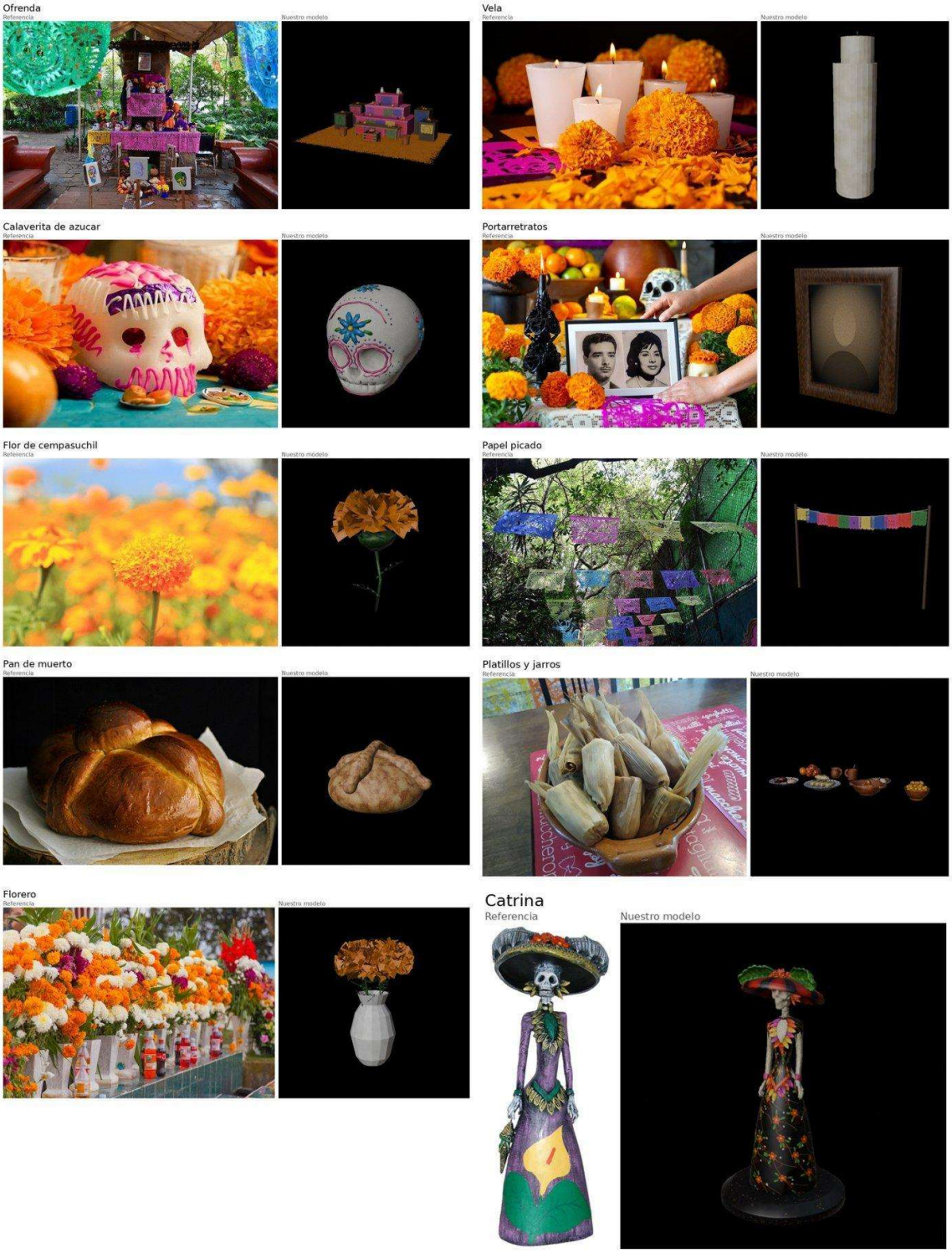


Figura 3: Foto de referencia (izquierda) y modelo terminado (derecha) de cada objeto.

Las fotos de páginas web tienen derechos reservados de sus autores, solo se usaron como referencia visual y aquí se cita su fuente (tabla 6). Las de Wikimedia Commons tienen licencia Creative Commons.

Objeto	Fuente de la foto	Autor o sitio	Licencia
Ofrenda	Ofrenda de Día de Muertos in Coyoacán	Inés Suárez R. (Wikimedia Commons)	CC BY-SA 4.0
Vela	Cuatro consejos para usar la flor de cempasúchil en la ofrenda	ADN40	Derechos reservados
Calaverita de azúcar	Cómo hacer calaveritas de azúcar	TV Azteca Jalisco	Derechos reservados
Portarretratos	Aura Frames quiere reunir más recuerdos familiares en las ofrendas	TechRadar México	Derechos reservados
Cempasúchil	6 cosas que debes saber de la flor de cempasúchil	Vive USA	Derechos reservados
Papel picado	Papel picado del Día de Muertos	Carmalvi (Wikimedia Commons)	CC BY-SA 4.0
Pan de muerto	Receta de pan de muerto	Directo al Paladar	Derechos reservados
Platillos	Día de Muertos: platillos	Ruptura 360	Derechos reservados
Florero	Tres flores indispensables para el altar de Día de Muertos	Meteored México	Derechos reservados
Catrina	Catrina sculpture	Dominik (Wikimedia Commons)	CC BY-SA 3.0

Tabla 6: Fuentes y licencias de las fotos de referencia.

Cada modelo distinto se carga una sola vez (de `assets/models/<modelo>/<modelo>.obj`) y se dibuja en cada posición con la matriz

$$\text{model} = T(x, y, z) \cdot R_y(\text{rotY}) \cdot S(\text{escala}).$$

Los datos salen de `assets/escena_layout.txt`, donde cada renglón coloca un objeto:

```
# modelo x y z rotY(grados) escala [tinte r g b]
ofrenda 0.000 0.000 -1.950 0.0 1.000
tumba -5.331 0.450 8.474 96.9 0.347 1.05 1.05 1.05
```

Las coordenadas son de OpenGL, en metros. El tinte opcional multiplica el color de la textura y sirve para pintar tumbas y velas de distintos colores con el mismo modelo. Cada renglón de vela se vuelve además una luz puntual.

Los modelos animados por jerarquía (músicos y niño) están separados en un OBJ por parte, con el origen de cada parte en su articulación, igual que el perro de la práctica *Máquina de estados*. El archivo `pivotes.txt` dice dónde está cada articulación y quién es su padre; los padres van antes que los hijos:

```
# parte pivote(x) pivote(y) pivote(z) padre
torso 0.0 0.6072 0.0 -
cabeza 0.0 1.0131 -0.0053 torso
brazoder -0.1155 0.9438 0.0 torso
antebrazoder -0.1327 0.7689 0.0026 brazoder
```

Por su parte, `assets/animaciones_layout.txt` da la posición y el giro de la banda y del niño que saluda, y `assets/models/banda/banda_layout.txt` acomoda a los siete músicos en arco respecto al centro de la banda.

5.2 Texturas y materiales

Las texturas son archivos PNG que se cargan con SOIL2 junto con cada OBJ y su `.mtl`. Para este proyecto, `Model.h` las carga en RGBA en lugar de RGB, porque el papel picado y las alas de las mariposas usan transparencia (canal alfa): los píxeles con alfa menor a 0.5 se descartan en el shader. Además, el color de la textura se multiplica por el tinte del layout y, si se ve la cara de atrás de una flor o un pétalo, se voltea la normal para que se ilumine igual. El cielo usa seis imágenes (right, left, top, bottom, front, back) cargadas en un `GL_TEXTURE_CUBE_MAP`.

5.3 Iluminación y ambientación

`lighting.frag` es el shader de *Fuentes de luz* con el modelo de **Phong**: Para cada fuente de luz, calcula la contribución ambiental, difusa y especular, combinándolas para obtener la iluminación final de la escena.

Luz	Tipo	Valores
Luna	Direccional	Dirección (−0,4, −0,8, 0,45), color azulado (0,45, 0,50, 0,75) por la intensidad de la luna (teclas + y −, de 0 a 2).
Velas	24 puntuales	Color cálido (1, 0,55, 0,2) por el parpadeo. Se mandan al shader las 24 velas más cercanas a la cámara (de 273).
Linterna	Spotlight en la cámara	Cono de 12° con borde suave hasta 18° y atenuación.

Tabla 7: Fuentes de luz de la escena.

Además del Phong del laboratorio, el shader agrega:

- **Fresnel.** Un brillo azulado de luna que aparece en los bordes de los objetos y se intensifica cuanto más se ven de perfil.
- **Niebla.** La niebla se vuelve más densa a medida que aumenta la distancia y se concentra más cerca del suelo. Su color se mezcla progresivamente con el de los objetos según la intensidad de la niebla
- **Caras dobles.** Si se ve la cara de atrás de una flor o un pétalo, se voltea la normal para que se ilumine igual.
- **Transparencia.** Los píxeles con alfa menor a 0.5 se descartan (huecos del papel picado y contorno de las alas).
- **Tinte por objeto.** El color de la textura se multiplica por el tinte del layout.

El **skybox** es un cubo con un `GL_TEXTURE_CUBE_MAP` de seis caras nocturnas. Se dibuja al final con `glDepthFunc(GL_LEQUAL)` y sin la traslación de la cámara, y en el horizonte se mezcla con el color de la niebla. Las **flamas** se dibujan con `lamp.frag` en color sólido, y su altura y color cambian con el mismo parpadeo de la luz de su vela.



Figura 4: Escena con Fresnel.



Figura 5: Escena sin Fresnel (tecla R).

6 Documentación del código fuente

6.1 Inicialización de OpenGL

Al iniciar, el programa crea la ventana con GLFW, inicia GLEW, configura el viewport y compila los cuatro shaders (lighting, lamp, skybox y viento). Después lee `escena_layout.txt`, carga cada modelo una sola vez (con sus versiones `_medio` y `_lod` si existen), calcula las matrices y esferas envolventes, ubica las velas y las animaciones (banda, reja, niño y mariposas por partes) y carga el skybox (flujo completo en la figura 1).

Si el programa no encuentra `assets/` en la carpeta actual (por ejemplo, al abrir el .exe con doble clic dentro de `build/`), cambia la carpeta de trabajo a la carpeta de arriba del ejecutable, donde están `assets/` y `shaders/`. Así el ejecutable no depende de la carpeta Debug ni de Visual Studio.

`Shader.h` y `Camera.h` se usan sin cambios. A `Mesh.h` y `Model.h` se les hicieron cambios pequeños, necesarios para dibujar una escena de este tamaño:

Archivo	Cambio	Por qué
`Mesh.h`	Las ubicaciones de los uniforms de las texturas se buscan una sola vez por shader y se guardan.	El original llamaba a `glGetUniformLocation` en cada dibujo; con 1,500 objetos eso bajaba mucho los FPS
`Mesh.h`	Guarda la caja que envuelve cada malla (mínimo, máximo).	Para calcular la esfera envolvente de cada objeto.
`Mesh.h` y `Model.h`	`Draw(Shader &shader)` recibe el shader por referencia.	Evita copiar el objeto Shader en cada dibujo.
`Model.h`	Funciones `Centro()` y `Radio()`.	Esfera envolvente del modelo.
`Model.h`	Las texturas se cargan en RGBA en lugar de RGB.	El papel picado y las alas de las mariposas usan el canal alfa.

Tabla 8: Cambios al código base del laboratorio.

6.2 Sistema de renderizado

En cada cuadro, el ciclo principal calcula `deltaTime`, lee el teclado y el ratón, avanza las animaciones y pasa las luces al shader (luna, las 24 velas más cercanas y linterna). Después descarta los objetos que quedan fuera del campo de visión y dibuja la escena (con nivel de detalle según la distancia), dibuja la banda, la reja, el niño y las mariposas por jerarquía, el papel picado con el shader del viento, las flamas de las velas con el shader `lamp` y, al final, el skybox, para terminar con `glfwSwapBuffers`.Para que esto corra con fluidez con más de 1,200 objetos aplicamos las optimizaciones de la tabla 9.

Técnica	Cómo funciona
Un modelo, muchas instancias	Cada modelo se carga una vez y se dibuja en todas sus posiciones. Los dibujos se ordenan por modelo para cambiar menos de texturas.
Matrices precalculadas	La matriz y la esfera envolvente de cada objeto se calculan una sola vez al iniciar.
Frustum culling	En cada cuadro se sacan los seis planos de la vista de projection · view; un objeto cuya esfera queda del lado de afuera de algún plano no se dibuja.
Niveles de detalle	A más de 4 m se usa la versión `_medio.obj` (personas, flores) y a más de 11 m la versión `_lod.obj`, con menos triángulos.
Niebla	Con niebla, los objetos pequeños (radio menor a 1.5 m) a más de 40 m no se dibujan porque ya no se ven.
Luces	Solo las 24 velas más cercanas a la cámara se mandan al shader.
Uniforms	`Mesh.h` guarda las ubicaciones de los uniforms en lugar de buscarlas en cada dibujo.

Tabla 9: Optimizaciones.

6.3 Cámara y perspectiva

La cámara es la de `Camera.h` del código base, sin cambios, y se mueve en primera persona (sección 6.4). La linterna es un spotlight que sale de la posición de la cámara y apunta hacia donde ve.Para descartar los objetos que quedan fuera de la vista, se comprueba su posición respecto a los límites de la cámara. El skybox se dibuja sin tener en cuenta el movimiento de la cámara, de modo que parece estar infinitamente lejos.

La cámara utiliza un campo de visión definido por camera.GetZoom(), con un plano cercano (near) de 0.1 y uno lejano (far) de 200.0 unidades. Su velocidad de movimiento se controla mediante la clase Camera y se aplica en función del tiempo transcurrido entre cuadros (deltaTime).

6.4 Manejo de entradas del usuario

`DoMovement()` mueve la cámara con W, A, S, D o las flechas y cambia la luz de la luna con + y −. `MouseCallback()` gira la cámara con el ratón. `KeyCallback()` guarda en el arreglo `keys` qué teclas están presionadas (para caminar) y, al presionar una tecla, cambia la bandera correspondiente (tabla 10); también cierra el programa con Esc.

Tecla	Variable	Efecto
F	`linternaOn`	Spotlight con o sin intensidad
V	`veladorasOn`	Luces puntuales y flamas
R	`fresnelOn`	Fuerza del Fresnel en 0 o 0.35
N	`nieblaOn`	Niebla en el shader y distancia de dibujo
+ / −	`lunaIntensidad`	Sube o baja 0.5 por segundo (en `DoMovement`)
1 a 5	`vientoOn`, `bandaOn`, `rejaAbierta`, `saludoOn`, `mariposasOn`	Animaciones

Tabla 10: Teclas que cambian la iluminación y las animaciones.

6.5 Funciones principales

Función	Qué hace
`CargaLayout(ruta)`	Lee `escena_layout.txt` y regresa la lista de instancias.
`MatrizModelo(i)`	Calcula $T \cdot R_y \cdot S$ de una instancia.
`Parpadeo(i, t)`	Controla la intensidad del parpadeo de las velas a lo largo del tiempo. Cada vela tiene un parpadeo ligeramente diferente para evitar que todas se iluminen al mismo ritmo
`CargaPartes(carpeta)`	Lee `pivotes.txt` y carga el OBJ de cada parte.
`DibujaPartes(...)`	Dibuja un modelo por partes: la matriz de cada parte es la de su padre por T(offset desde el padre) $\cdot$ R(ángulos de la parte).
`PoseMusico(...)`	Calcula los ángulos de cada parte de un músico en el tiempo t (animación 2).
`CatmullRom(p0, p1, p2, p3, u)` [3]	Calcula la posición de la mariposa sobre una trayectoria curva (animación 5).
`PosMariposa(m, u)`	Posición de la mariposa `m` en el tramo actual de sus keyframes.
`AvanzaAnimaciones(dt)`	Avanza el estado de las cinco animaciones una vez por cuadro.
`DoMovement()`	Mueve la cámara con W A S D o flechas y cambia la luz de la luna con + y −.
`KeyCallback(...)`	Prende y apaga luces y animaciones (F, V, R, N, 1 a 5) y cierra con Esc.
`MouseCallback(...)`	Gira la cámara con el ratón.
`LoadCubemap(caras)`	Carga las seis imágenes del skybox en un `GL_TEXTURE_CUBE_MAP`.

Tabla 11: Funciones principales de `main.cpp`.

7 Sistema de animaciones

Todas las animaciones se calculan en el código con el tiempo de cada cuadro (`deltaTime`), así que van a la misma velocidad en cualquier computadora. Las sencillas siguen la idea de la práctica *Animación básica* y las complejas se basan en *Máquina de estados* y *KeyFrames*.

7.1 Animación sencilla 1: viento en el papel picado (tecla 1)

La variable `fuerzaViento` aumenta o disminuye gradualmente entre 0 y 1, según si el viento está activado. El movimiento del papel picado se calcula en el shader `viento.vs`, que desplaza ligeramente cada vértice mediante ondas.

El movimiento es más intenso en la parte inferior y central de las tiras, mientras que los extremos sujetos a los postes permanecen prácticamente fijos. Además, cada tira tiene un movimiento ligeramente diferente para evitar que todas se muevan al mismo ritmo.

7.2 Animación sencilla 2: banda tocando (tecla 2)

Cada músico es un modelo jerárquico por partes (cuerpo, torso, brazos, cabeza, instrumento) dibujado con `DibujaPartes`. `PoseMusico` calcula sus ángulos con el tiempo a 120 golpes por minuto: el cuerpo se mece de lado a lado ($\pm 6°$) y rebota, el torso se inclina, la tarola y la tambora golpean con los brazos (hasta $35°$), el trombón saca la vara y su brazo la sigue, y los instrumentos de viento suben un poco. `fuerzaBanda` va de 0 a 1 en medio segundo y multiplica todos los ángulos, para que empiecen y dejen de tocar sin cortes.

7.3 Animación sencilla 3: reja de la entrada (tecla 3)

Cada hoja de la reja es un OBJ con su origen en la bisagra. `tReja` avanza o regresa entre 0 y 3 segundos según `rejaAbierta`, y el ángulo se suaviza para que arranque y frene despacio.



Figura 6: La reja abierta.

7.4 Animación compleja 1: niño que saluda (tecla 4)

Se basa en la práctica *Máquina de estados*. El niño es un modelo por partes y su comportamiento depende del estado en que está (figura 7 y tabla 12).

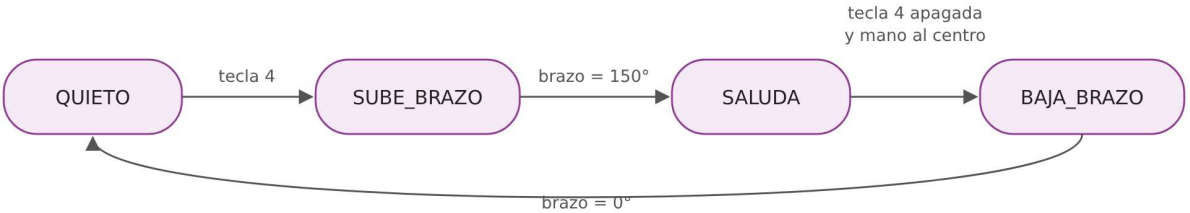


Figura 7: Máquina de estados del niño que saluda.

Estado	Qué hace
`QUIETO`	Brazo abajo. Si se activa la tecla 4 pasa a `SUBE_BRAZO`.
`SUBE_BRAZO`	El brazo derecho sube 150° por segundo y la cabeza se inclina hasta 8°. Al llegar a 150° pasa a `SALUDA`.
`SALUDA`	El antebrazo va de −25° a 25° y regresa, cambiando de dirección con la bandera `manoDerecha`. Si se apaga la tecla y la mano está cerca del centro, pasa a `BAJA_BRAZO`.
`BAJA_BRAZO`	El brazo baja 120° por segundo y la cabeza regresa. Al llegar a 0° vuelve a `QUIETO`.

Tabla 12: Estados del niño que saluda.

Es compleja porque no es una sola transformación: combina estados, condiciones de cambio y la jerarquía hombro, antebrazo y cabeza.

7.5 Animación compleja 2: mariposas monarca (tecla 5)

Se basa en la práctica *KeyFrames*: cada una de las tres mariposas tiene un arreglo de tipo `FRAME` con ocho posiciones alrededor de la ofrenda, y las variables `playIndex` (keyframe donde empieza el tramo) y `tramo` (de 0 a 1) avanzan con el tiempo: cada tramo dura 1 segundo y al terminar el último se regresa al primero.

A diferencia de la práctica, la posición entre dos keyframes no es una línea recta: se usa una curva de **Catmull-Rom** [3], que pasa por todos los keyframes y usa el anterior y el siguiente para que la trayectoria sea suave:

$$P(u) = \tfrac{1}{2}\left[2P_1 + (P_2 - P_0)\,u + (2P_0 - 5P_1 + 4P_2 - P_3)\,u^2 + (3P_1 - P_0 - 3P_2 + P_3)\,u^3\right]$$

Con la dirección de la curva se calcula hacia dónde voltea la mariposa y si va subiendo o bajando. La mariposa es un modelo jerárquico de tres partes: el cuerpo y dos alas hijas del cuerpo que giran sobre el eje Z entre 10° y 70° seis veces por segundo, y el cuerpo sube y baja un poco con cada aleteo. Es compleja porque combina keyframes, una trayectoria curva, la orientación calculada con la tangente y la jerarquía de las alas.



Figura 8: Mariposas monarca volando alrededor de la ofrenda.

8 Resultados

8.1 Evidencias del resultado final

La figura 9 compara la imagen de referencia de la ofrenda con su recreación en OpenGL. Otras evidencias están en las figuras 3 (objeto por objeto), 7 y las capturas de las animaciones de la sección anterior.

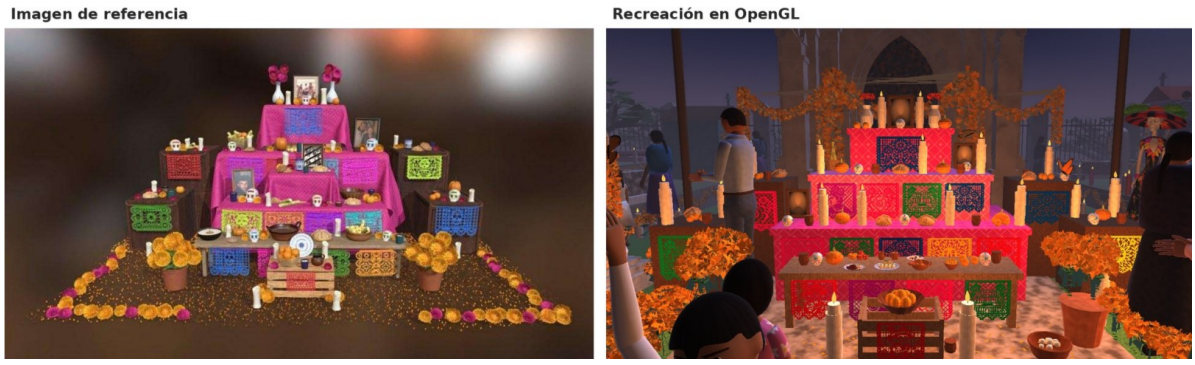


Figura 9: Imagen de referencia y recreación en OpenGL.

9 Conclusiones

Logramos recrear un panteón mexicano en Día de Muertos con una ofrenda tradicional parecida a la imagen de referencia, basándonos en los diferentes códigos base del laboratorio. El proyecto nos permitió juntar en un solo programa todos los temas del curso: carga de modelos con texturas, iluminación de Phong con varias fuentes de luz, cube mapping para el cielo, modelado jerárquico y los tres tipos de animación vistos en las prácticas.

Lo más difícil fue que la escena se viera llena y realista sin que el programa se volviera lento. Leer la escena de un archivo de datos nos permitió colocar muchos modelos sin tener que repetir código, y técnicas como la eliminación de objetos fuera del campo de visión, los niveles de detalle y mandar solo las velas más cercanas al shader hicieron que se pudiera recorrer con fluidez.

En las animaciones aprendimos que para hacer una animación compleja bastó con combinar bien las ideas de las prácticas: una máquina de estados con jerarquía para el niño, y keyframes con una curva suave, orientación calculada y aleteo para las mariposas. Subir avances diarios a GitHub nos ayudó a probar cada parte antes de seguir y dejó documentado todo el desarrollo.

Referencias

[1] Espinoza Urzúa, E. *Proyecto Final, Laboratorio de Computación Gráfica*. 2026.

[2] Prácticas del laboratorio: *Fuentes de luz*, *Animación básica*, *Máquina de estados* y *KeyFrames*. Código base proporcionado por el profesor.

[3] Boost C++ Libraries. (s.f.). *Catmull-Rom splines*. Boost.Math Documentation. `https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/catmull_rom.html`

[4] Six, J. (2021, septiembre). *Frustum Culling*. LearnOpenGL. `https://learnopengl.com/Guest-Articles/2021/Scene/Frustum-Culling`

[5] Unity Technologies. (s. f.). *Level of Detail (LOD)*. Unity. `https://unity.com/glossary/level-of-detail`

[6] Quílez, I. (s. f.). *Smoothsteps*. Inigo Quílez. `https://iquilezles.org/articles/smoothsteps/`

[7] Vries, J. (2017). *PBR Theory*. LearnOpenGL. `https://learnopengl.com/PBR/Theory`

[8] cppreference.com. (s. f.). *std::partial_sort*. C++ Reference. `https://en.cppreference.com/w/cpp/algorithm/partial_sort`

[9] Fotos de referencia de los objetos: ver la tabla 6.

[10] Documentación de las herramientas usadas (OpenGL, GLFW, GLEW, GLM, assimp, SOIL2 y Blender): ver la tabla 1.

10 Repositorio del proyecto

El proyecto se encuentra alojado en GitHub, donde se puede consultar el código fuente, los recursos utilizados y el historial de cambios realizados durante el desarrollo.

Enlace al repositorio:
`https://github.com/VvyasservV/Ofrenda-dia-de-muertos-laboratorio-de-CG`

El video explicativo del proyecto está en YouTube:
`https://youtu.be/TYJ01hJs5oU`